HOST A WEBSITE ON AMAZON S3

By Rushikesh Kulkarni | November 2026

Introducing Today's Project!

Project overview

Today, I am here to host a static website on AWS using Amazon S3. I will create a globally unique S3 bucket, upload my website files, configure static website hosting, and make the objects public using ACLs so that anyone on the internet can access my website.

Tools and concepts

During this project, I gained hands-on experience with the following key cloud services and technical concepts:

Amazon S3: Understanding how cloud-based object storage works, creating an S3 bucket, and the absolute requirement for bucket names to be globally unique.

Static Website Hosting: Setting up an S3 bucket to work as a web server to generate a unique, public website endpoint URL.

AWS Regions: Learning how choosing a local region, like Sydney (ap-southeast-2), helps reduce latency and increases website load speed.

Permissions & ACLs: Managing file-level security, resolving 403 Forbidden errors, and adjusting Block Public Access settings to safely make assets public.

S3 Bucket Policies: Writing JSON policies to enforce advanced security rules, such as using an explicit Deny rule and wildcard (*) elements to safeguard core website files.

Time, challenges, and wins

It took me approximately 30 to 45 minutes to complete this entire project. During this time, I successfully created an S3 bucket, uploaded my website files, enabled static website hosting, resolved the 403 Forbidden error using ACLs, and configured a custom JSON bucket policy for the secret security mission.

❤ Thanks for doing this project!

Honestly, I love it! I feel so happy and filled with gratification after completing this project. Thanks to NextWork, my learning journey has been amazing, and hats off to the team for making this platform so fun and rewarding❤.

How I Set Up an S3 Bucket

What I did in this step

"In this step, I will open Amazon and create an S3 bucket as a storage container so I can start storing my static website files in the cloud.

How long it took to create the bucket

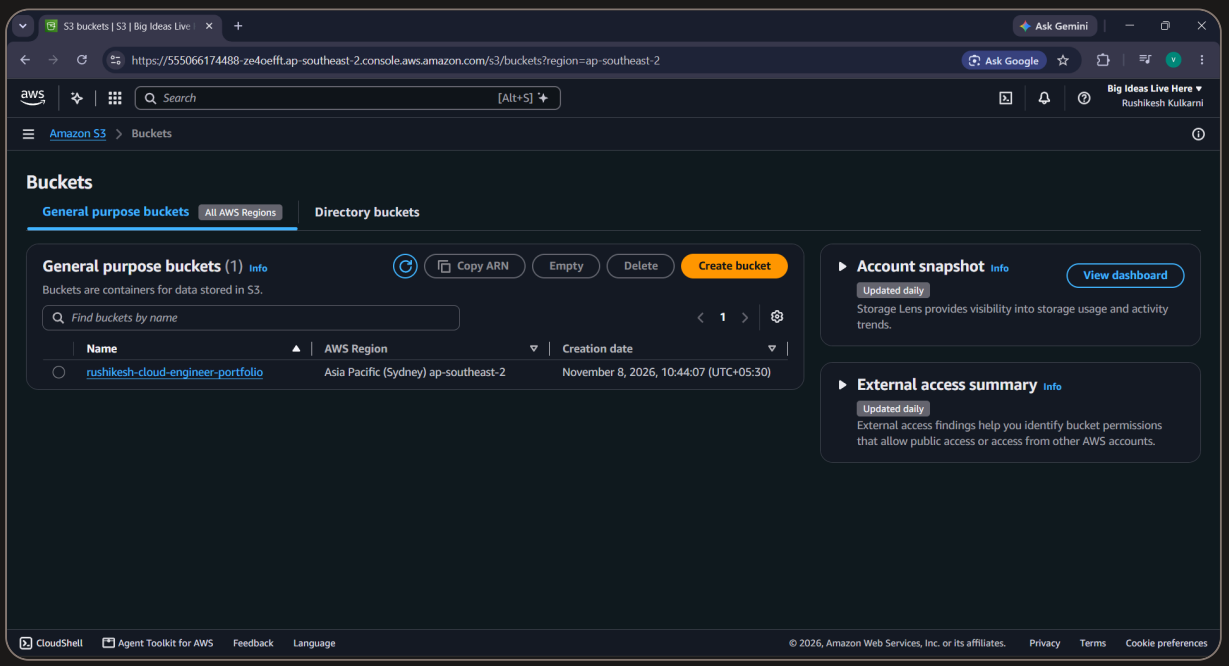
Creating an S3 buclt took only a few seconds (under 10 seconds) to create the S3 bucket. Once I clicked the create button, AWS provisioned and made the storage space active almost instantly.

Region selection

I was selected the default Asia Pacific (Sydney) ap-southeast-2 AWS Region because it was automatically selected as the default region for my account when I created my bucket.

Understanding bucket name uniqueness

It means that no two in the entire world can share the exact same name, across any or region. Because each bucket is assigned a public website address, the name must be to prevent domain name conflicts.



Upload Website Files to S3

What I did in this step

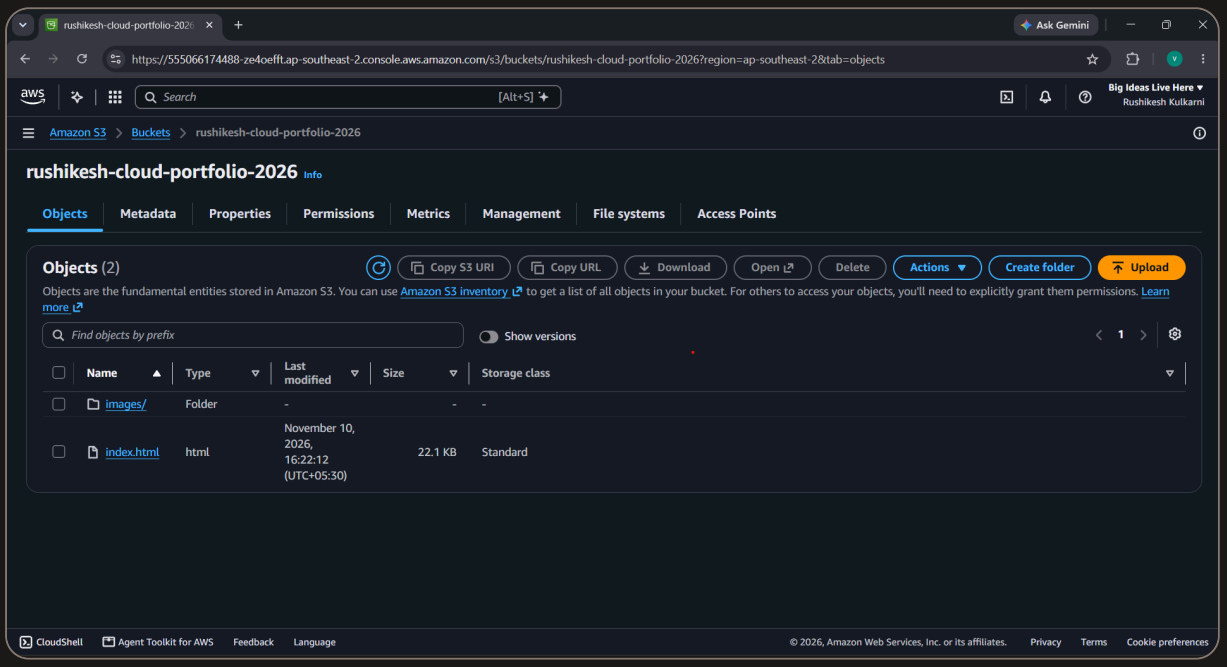
In this step, I will download the index.html file and the website's image files to my local computer, and then upload them into my Amazon S3 bucket. I am doing this to store all of my website's content files in the cloud so they can be loaded by visitors.

Files I uploaded

I uploaded the index.html file (the structural blueprint of my webpage) and the unzipped assets folder containing the images and media files for my rushikesh-cloud-portfolio-2026 portfolio website.

How the files work together

The index.html file provides the core structure and code of the webpage, while the assets folder contains the images and files that the HTML code references. You need both because the HTML file cannot display your portfolio images or layout correctly if those referenced asset files are missing from the S3 bucket.



Static Website Hosting on S3

What I did in this step

In this step, I will enable static website hosting on my Amazon S3 bucket and set index.html as my index document. I am doing this to convert my S3 storage bucket into a web server that can generate a unique public endpoint URL for visitors to access my portfolio website.

Understanding website hosting

Website hosting means storing your website's files—like HTML documents, CSS files, images, and other assets—on a secure web server connected to the

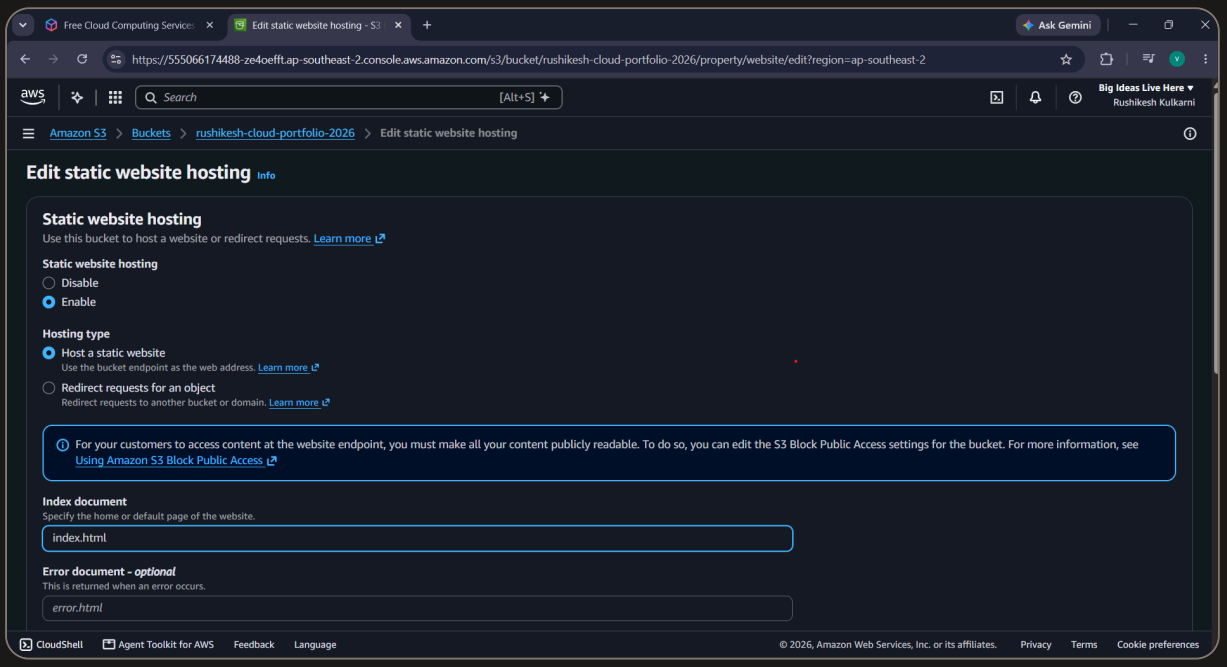
internet. This allows anyone around the world to access and view your website by entering its web address into a browser.

How I enabled website hosting

I enabled hosting by opening my Amazon S3 bucket in the AWS Console, navigating to the Properties tab, and scrolling down to the Static website hosting section. I clicked Edit, selected Enable, specified index.html as the index document, and saved my changes.

Access Control Lists (ACLs)

An Access Control List (ACL) is a set of rules that defines who has permission to access and manage files inside an S3 bucket. In this step, I enabled ACLs so that I can control access permissions for my individual website files and make them publicly accessible to visitors.



Bucket Endpoints

Understanding bucket endpoint URLs

When I first visited the bucket endpoint URL, I saw a 403 Forbidden error page. The reason for this error is that all files uploaded to Amazon S3 are private by default to keep the account secure. Since we have not made our HTML and image files public yet, S3 blocks us from viewing them.

What I saw when I tested the endpoint

I saw a 403 Forbidden error page displaying an 'Access Denied' message. This happened because Amazon S3 keeps all uploaded files private by default to maintain cloud security, so public access to our website files is blocked until we explicitly configure permissions to make them public.

403 Forbidden

- Code: AccessDenied
- Message: Access Denied
- RequestId: R05PAMHQ8AW33M7W
- HostId: /MGlJ6Imk4Xsb8FBuFdVCZ3u/Q1fxCj4HUORUQQ6GPJFOOaliFwBFU/Cg7H8wb9fTy2Sb2EjyN1zDqfMKwdwVmFe+ftCzFbN

Success!

What I did in this step

In this step, I will make the uploaded website files public using object-level Access Control Lists (ACLs). I am doing this to resolve the 403 Forbidden error and allow visitors to successfully load and view my portfolio website over the internet.

How I resolved the 403 error

I resolved the 403 Forbidden error by going to the Objects tab inside my Amazon S3 bucket. I selected both the index.html file and the assets folder, clicked the Actions dropdown menu, selected Make public using ACL, and clicked Make public to confirm the changes.

□ What were the key services and concepts you learnt in this project?

In this project, I learned how to use key cloud services and security configurations to host a web application. The core services and concepts include:

Amazon S3: Storing website assets in a cloud-based object storage service and understanding the requirement for globally unique bucket naming.

Static Website Hosting: Configuring S3 to act as a lightweight web server and generating a public bucket website endpoint URL.

AWS Regions: Learning how regional selection (such as Sydney ap-southeast-2) impacts latency, performance, and website load speeds for local users.

Access Control & ACLs: Resolving 403 Forbidden errors by disabling Block Public Access settings and using object-level Access Control Lists to safely expose files to the public.

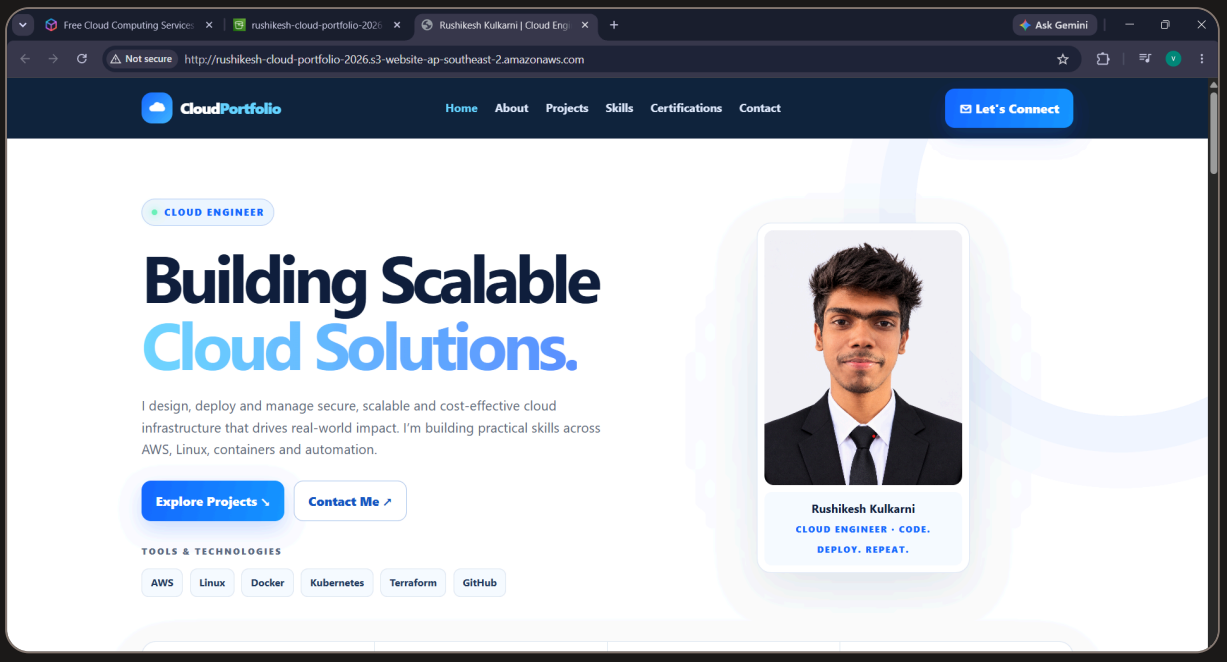
S3 Bucket Policies: Writing JSON policies to implement granular security rules, utilizing explicit Deny effects, and leveraging the wildcard (*) character to manage permissions globally or protect entire directories.

□ How long did it take you to complete this project?

It took me approximately 30 to 45 minutes to complete this entire project. This time was spent creating and configuring the Amazon S3 bucket, downloading and uploading my portfolio files, enabling static website hosting, resolving the 403 Forbidden error, and writing a custom JSON bucket policy for the secret mission.

❤ Thanks for doing this project!

Honestly, I love it! It is already an amazing platform and makes me feel so happy and excited to learn. Keeping the interactive challenges, secret missions, and fast-paced guides coming is exactly what will make it even better!



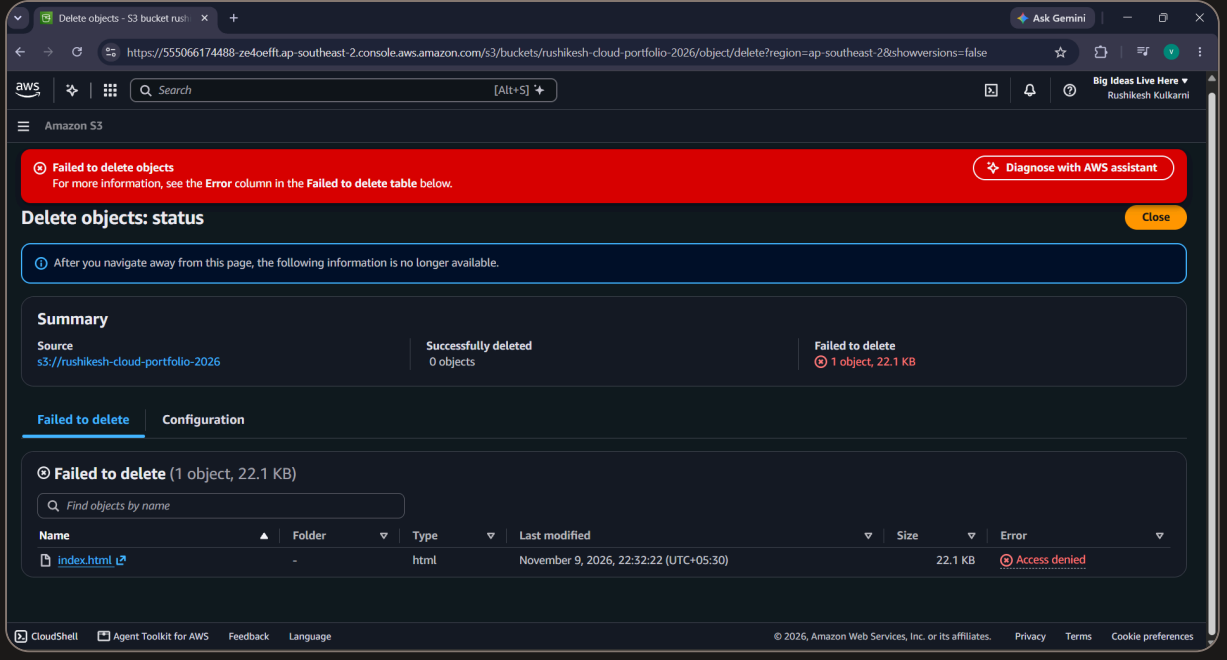
Bucket Policies

What I did in this extension

In this secret mission, I am going to set up a custom S3 bucket policy that explicitly denies anyone—including myself—from deleting the index.html file. I am doing this to learn how to write JSON-based access policies, which provide much more granular security and control over bucket resources than standard ACLs.

Understanding bucket policies

In this secret mission, I am going to set up a custom S3 bucket policy that explicitly denies anyone—including myself—from deleting the index.html file. I am doing this to learn how to write JSON-based access policies, which provide much more granular security and control over bucket resources than standard ACLs.



What my bucket policy does

My bucket policy uses an explicit Deny effect with a wildcard principal ("Principal": "*") to ensure that the delete restriction applies to all users globally. While targeting (index.html)protects that specific file, using a wildcard resource ending in (/*) would block the deletion of every file in the bucket to protect all my portfolio assets at once. I successfully verified this policy in action by attempting to delete my file, which AWS immediately blocked with a 'Failed to delete' Access Denied error.